

Module 15 - chapter 16

Project: Run Docker applications - Part 1

The project files used in this lecture can be found here: <https://gitlab.com/twn-devops-bootcamp/latest/15-ansible/terraform-learn/-/tree/feature/deploy-to-ec2>

Ansible & Docker

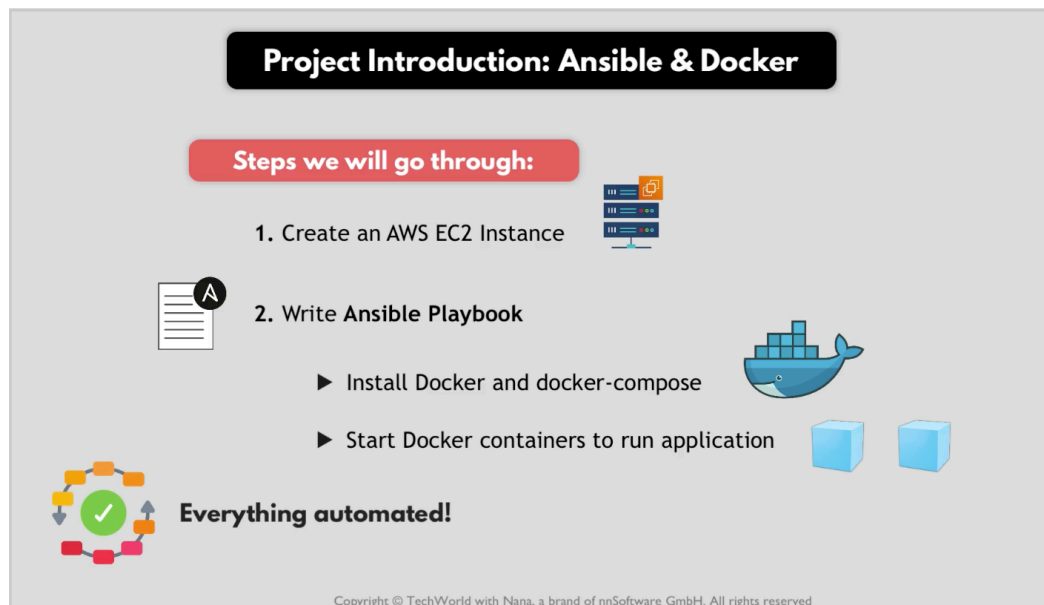
Project Overnieuw

We will write an ansible playbook to do the below:

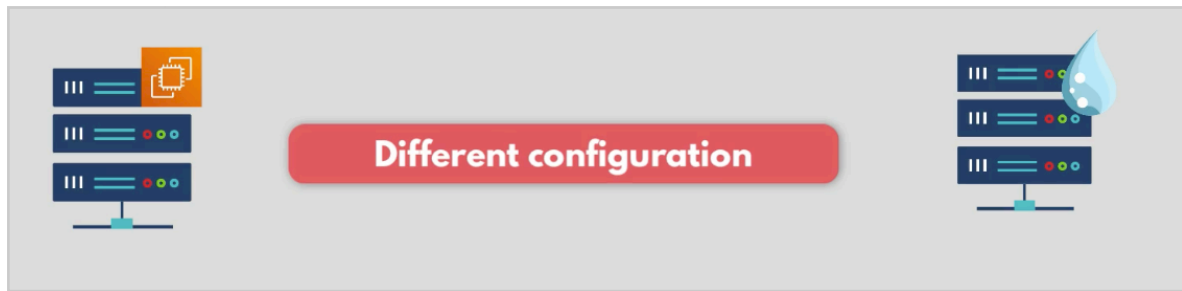
- Install Docker
- Install Docker compose

On a AWS EC2 server.

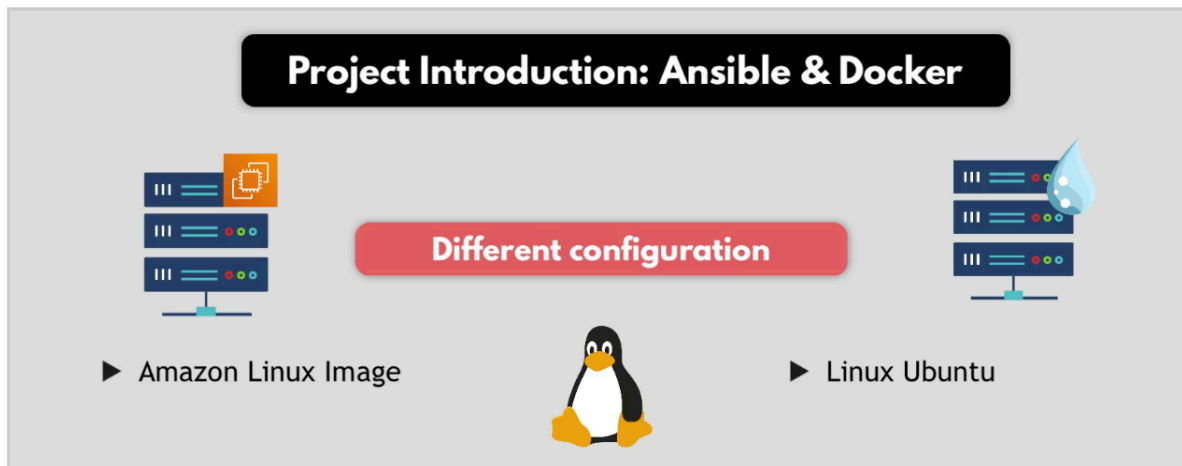
- Using a docker compose file from one of our previous projects we will start the containers on that EC2 server.



Configuration for the AWS EC2 server is different to the droplet configuration.



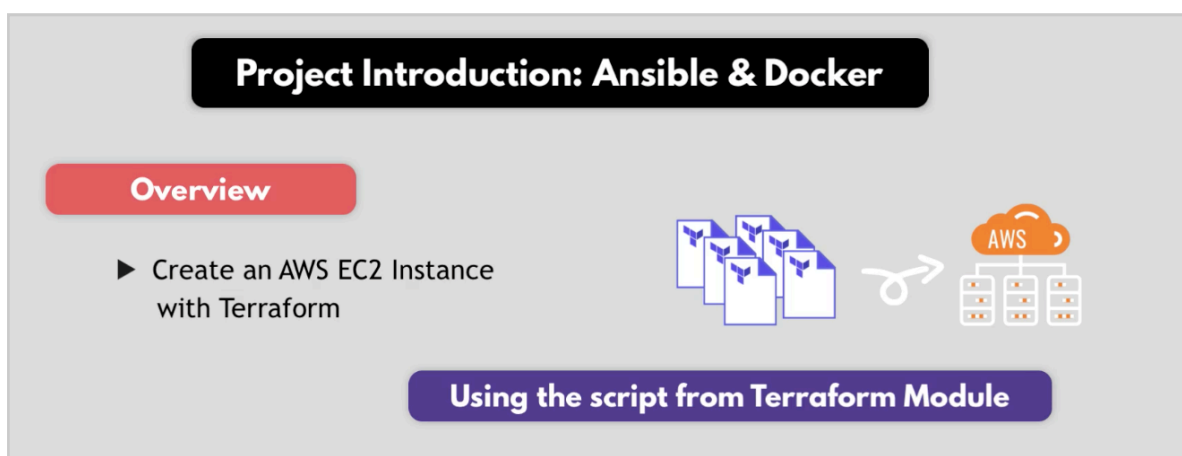
Their OS are different.



EC2 server uses Amazon Linux, the droplet uses Linux Ubuntu.

So installing packages and other things will be different.

1. To create the EC2 instance we will use Terraform so we do it automatically. We will use a tf file from a previous project



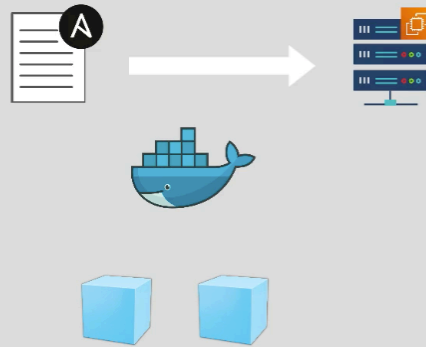
2. Connect EC2 instance to Ansible and create a playbook to install docker and docker compose, copy docker-compose file to server and start docker containers

- ▶ Configure Inventory file to connect to AWS EC2 Instance

- ▶ Install Docker and docker-compose

- ▶ Copy docker-compose file to server

- ▶ Start Docker containers



Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

Create AWS EC2 Instance

I copy the code from Module 12 - chapter 14 https://github.com/AstridCaballero/Module_12_Terraform/blob/Module_12/Terraform_chapter-14/main.tf

And create a new branch https://github.com/AstridCaballero/Module_15_Ansible/tree/Module_15/Ansible_chapter-16_Terraform

And my branch doesn't have terraform.tfvars committed to the remote branch, so I found them in my notes and here they are:

```
vpc_cidr_block = "10.0.0.0/16"
subnet_cidr_block = "10.0.10.0/24"
avail_zone      = "eu-west-2a"
env_prefix      = "dev"
my_ip           = "140.228.47.252/32"
instance_type   = "t2.small"
public_key_location = "/Users/astrid/.ssh/id_ed25519.pub"
image_name      = "al2023-ami-2023*-kernel-*-x86_64"
```

main.tf	terraform.tfvars
5 variable vpc_cidr_block {}	1 vpc_cidr_block = "10.0.0.0/16"
6 variable subnet_cidr_block {}	2 subnet_cidr_block = "10.0.10.0/24"
7 variable avail_zone {}	3 avail_zone = "eu-west-2a"
8 variable env_prefix {}	4 env_prefix = "dev"
9 variable my_ip {}	5 my_ip = "140.228.47.252/32"
10 variable instance_type {}	6 instance_type = "t2.small"
11 variable public_key_location {}	7 public_key_location = "/Users/astrid/.ssh/id_ed25519.pub"
12 variable image_name {}	8 image_name = "al2023-ami-2023*-kernel-*-x86_64"
13	

In the previous terraform projects we used a shell script

```
main.tf x entry-script.sh x
1 1 ▶ #!/bin/bash
2 2 sudo dnf update -y && sudo dnf install -y docker
3 3 sudo systemctl start docker
4 4 sudo usermod -aG docker ec2-user
5 5 docker run -p 8080:80 nginx
```

Here we don't need it, because we only want to provision the server, we don't want to execute anything inside them yet. We want to use Ansible to configure the EC2 server.

So we remove the shell file and we also remove lines 139 and 141 📌 when creating the resource "aws_instance"

```
127
128 resource "aws_instance" "myapp-server" {
129     ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
130     instance_type = var.instance_type
131
132     subnet_id = aws_subnet.myapp-subnet-1.id
133     vpc_security_group_ids = [aws_default_security_group.default-sg.id]
134     availability_zone = var.avail_zone
135
136     associate_public_ip_address = true
137     key_name = aws_key_pair.ssh-key.key_name
138
139     user_data = file("entry-script.sh")
140
141     user_data_replace_on_change = true
142
143     tags = {
144         Name = "${var.env_prefix}-server"
145     }
146 }
```

Now we can execute the configuration
-> terraform init


```

) terraform init
Initializing provider plugins found in the configuration...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.58.0...
- Installed hashicorp/aws v6.58.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

```

-> terraform apply

(I had to run it twice, first run created everything but the EC2 instance, second run I edited t2.small to t3.small)

```

aws_instance.myapp-server-one: Creating...
aws_instance.myapp-server-one: Still creating... [00m10s elapsed]
aws_instance.myapp-server-one: Creation complete after 13s [id=i-090a1d6381ea14871]

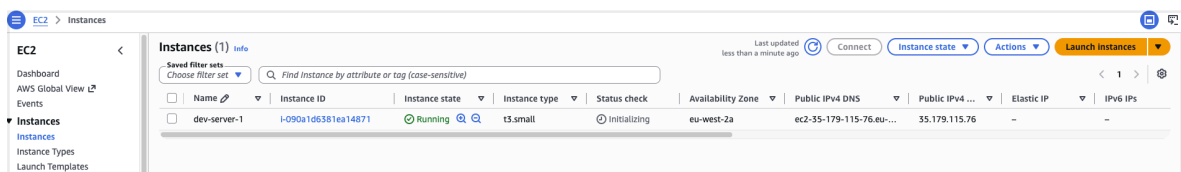
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

aws-ami_id = "ami-01c952cfc86b7870d"
ec2-public_ip_1 = "35.179.115.76"

```

And we have the EC2 instance running



Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
dev-server-1	i-090a1d6381ea14871	Running	t3.small	Initializing	eu-west-2a	ec2-35-179-115-76.eu-...	35.179.115.76	--	--

Now we have a server that it has now configuration yet

Overview

✓

- ▶ Create an AWS EC2 Instance with Terraform
- ▶ Configure Inventory file to connect to AWS EC2 Instance
- ▶ Install Docker and docker-compose
- ▶ Copy docker-compose file to server
- ▶ Start Docker containers

```

aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [10s elapsed]
aws_instance.myapp-server: Still creating... [20s elapsed]
aws_instance.myapp-server: Creation complete after 22s [id=i-084dcf4e54e402122]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

aws-ami_id = "ami-03cceb10496c25679"
ec2-public_ip = "3.75.173.238"
nana@macbook /Users/nana/terraform [feature/deploy-to-ec2]

```

Now Ansible will take from here ✓

Adjust Inventory File

I create a new branch from Module 15 - chapter 15 https://github.com/AstridCaballero/Module_15_Ansible/tree/Module_15_Ansible_chapter-16_ansible

From the terraform execution:

```

aws_instance.myapp-server-one: Creating...
aws_instance.myapp-server-one: Still creating... [00m10s elapsed]
aws_instance.myapp-server-one: Creation complete after 13s [id=i-090a1d6381ea14871]

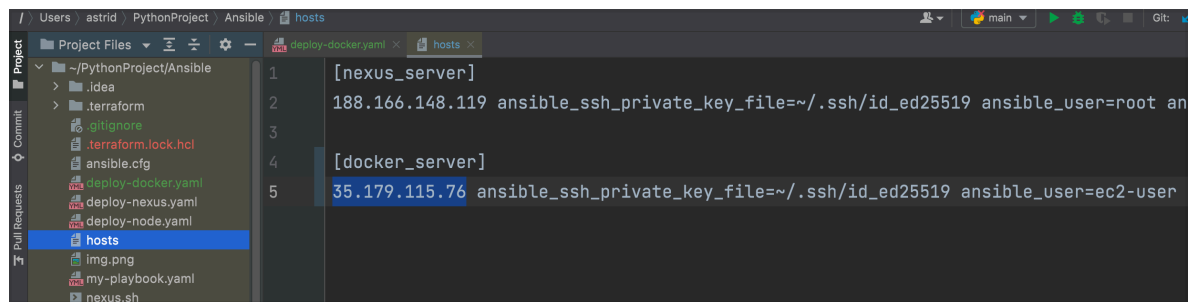
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

aws-ami_id = "ami-01c952cfc86b7870d"
ec2-public_ip_1 = "35.179.115.76"

```

I copy and I add the IP address of the EC2 instance to the 'hosts' file



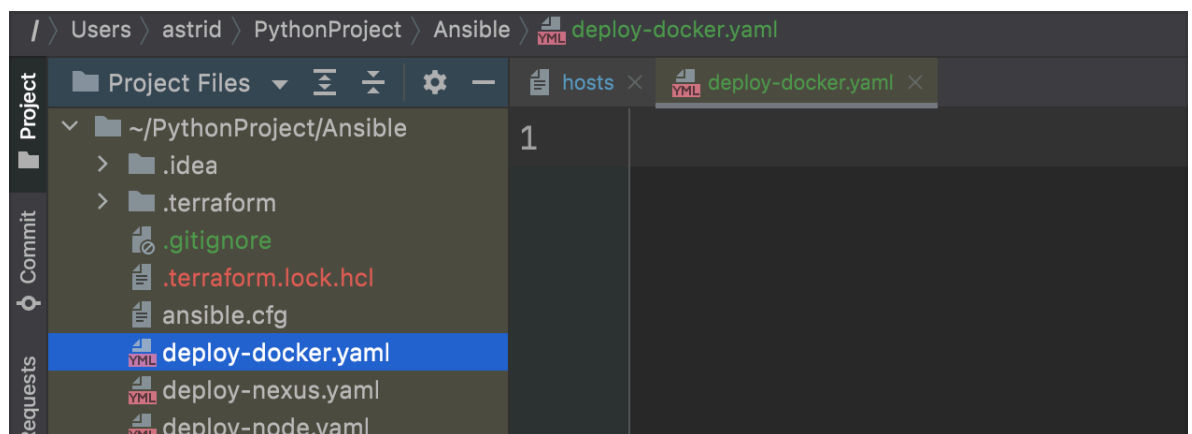
```
1 [nexus_server]
2 188.166.148.119 ansible_ssh_private_key_file=~/.ssh/id_ed25519 ansible_user=root an
3
4 [docker_server]
5 35.179.115.76 ansible_ssh_private_key_file=~/.ssh/id_ed25519 ansible_user=ec2-user
```

And for Ec2 instances the user is ec2-user so I use that value for ansible_user:



```
4 [docker_server]
5 35.179.115.76 ansible_ssh_private_key_file=~/.ssh/id_ed25519 ansible_user=ec2-user ansible_python_int
```

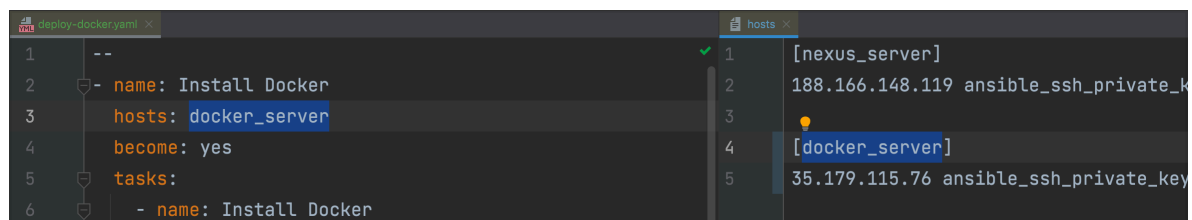
And let's create a new playbook 'deploy-docker.yaml'



```
1
```

Write first play

Install docker



```
1 --
2 - name: Install Docker
3   hosts: docker_server
4   become: yes
5   tasks:
6     - name: Install Docker
```

Now in the EC2 instance the package manager tool is not 'apt', so if I ssh into the ec2 instance and check for 'apt' it can't be found

```

> ssh ec2-user@35.179.115.76
The authenticity of host '35.179.115.76 (35.179.115.76)' can't be established.
ED25519 key fingerprint is SHA256:br4jN6Y2eL2gwss4BmF0800QdYEFMTZf3NEQTp3bJz4.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '35.179.115.76' (ED25519) to the list of known hosts.

      #_
     ~\_ #####_      Amazon Linux 2023
    ~ ~ \_#####\
    ~ ~ \_###|
    ~ ~ \#/_____|
    ~ ~  V~'  ' ->      https://aws.amazon.com/linux/amazon-linux-2023
    ~ ~
    ~ ~ _.-.
    ~ ~ _/_/_/
    ~ ~ _/m/'

[ec2-user@ip-10-0-10-56 ~]$ apt
-bash: apt: command not found
[ec2-user@ip-10-0-10-56 ~]$

```

And if I try 'yum' we can see is installed

```

    _/m/
[ec2-user@ip-10-0-10-56 ~]$ apt
-bash: apt: command not found
[ec2-user@ip-10-0-10-56 ~]$ yum
usage: yum [options] COMMAND

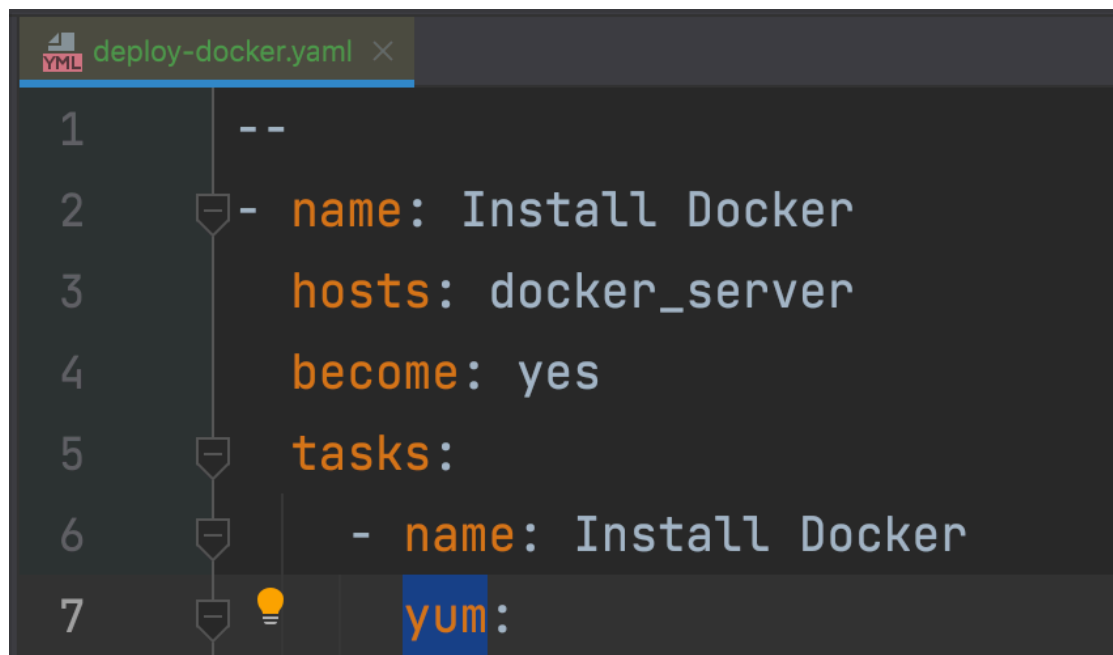
List of Main Commands:

alias                List or create command aliases
autoremove           remove all unneeded packages that were originally installed
check                check for problems in the packagedb
check-update          check for available package upgrades
clean                remove cached data
deplist              [deprecated, use repoquery --deplist] List package's dependencies
distro-sync           synchronize installed packages to the latest available
downgrade            Downgrade a package
group                display, or use, the groups information
help                 display a helpful usage message
history              display, or use, the transaction history

```

So yum is the package manager tool in EC2 instance!

And now our play looks like this:



```
1  --
2  - name: Install Docker
3    hosts: docker_server
4    become: yes
5    tasks:
6      - name: Install Docker
7        yum:
```

And in ansible 'yum' module docs

https://docs.ansible.com/projects/ansible/devel/collections/ansible/builtin/yum_repository_module.html

We can find example of how to use it.

Examples

```
- name: Add repository
  ansible.builtin.yum_repository:
    name: epel
    description: EPEL YUM repo
    baseurl: https://download.fedoraproject.org/pub/epel/$releasever/$basearch/

- name: Add multiple repositories into the same file (1/2)
  ansible.builtin.yum_repository:
    name: epel
    description: EPEL YUM repo
    file: external_repos
    baseurl: https://download.fedoraproject.org/pub/epel/$releasever/$basearch/
    gpgcheck: no

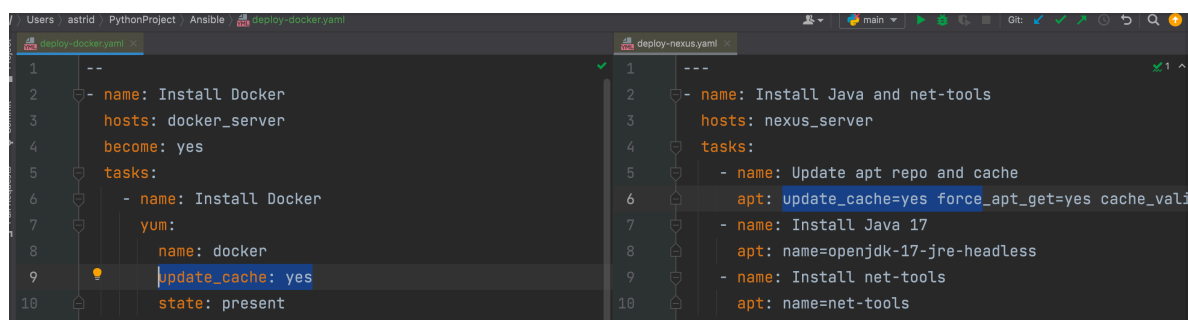
- name: Add multiple repositories into the same file (2/2)
  ansible.builtin.yum_repository:
    name: rpmforge
    description: RPMforge YUM repo
    file: external_repos
    baseurl: http://apt.sw.be/redhat/el7/en/$basearch/rpmforge
    mirrorlist: http://mirrorlist.repoforge.org/el7/mirrors-rpmforge
    enabled: no

# Handler showing how to clean yum metadata cache
- name: yum-clean-metadata
  ansible.builtin.command: yum clean metadata

# Example removing a repository and cleaning up metadata cache
- name: Remove repository (and clean up left-over metadata)
  ansible.builtin.yum_repository:
    name: epel
    state: absent
  notify: yum-clean-metadata

- name: Remove repository from a specific repo file
  ansible.builtin.yum_repository:
    name: epel
    file: external_repos
    state: absent
```

And we want to install docker using 'yum' and do something similar to what we did for 'deploy-nexus.yaml'



present to install a package, **absent** to uninstall 🙌

Currently we are using 'ec2-user' which is an 'admin' user not 'root' user. And to install a package we need to use a 'root' user:

```
[ec2-user@ip-10-0-10-186 ~]$ yum install docker
Error: This command has to be run with superuser privileges (under the root user on most systems).
[ec2-user@ip-10-0-10-186 ~]$
```



So we need to tell ansible to run the install command using 'root' user instead of 'ec2-user' and for that we use 'become'

```
1  --
2  - name: Install Docker
3    hosts: docker_server
4    become: yes
5    become_user: root
6    tasks:
7      - name: Install Docker
8        yum:
9          name: docker
10         update_cache: yes
11         state: present
```

And the default value of 'become_user' is root so we actually don't need line 5 🙅 as it is redundant

```
YML deploy-docker.yaml x
1      --
2      - name: Install Docker
3        hosts: docker_server
4        become: yes
5        tasks:
6          - name: Install Docker
7            yum:
8              name: docker
9              update_cache: yes
10             state: present
```

Let's execute the playbook

-> ansible-playbook deploy-docker.yaml

```
TASK [Gathering Facts] *****
[ERROR]: Task failed: Action failed: The following modules failed to execute: ansible.legacy.setup.

Task failed: Action failed.

<<< caused by >>>

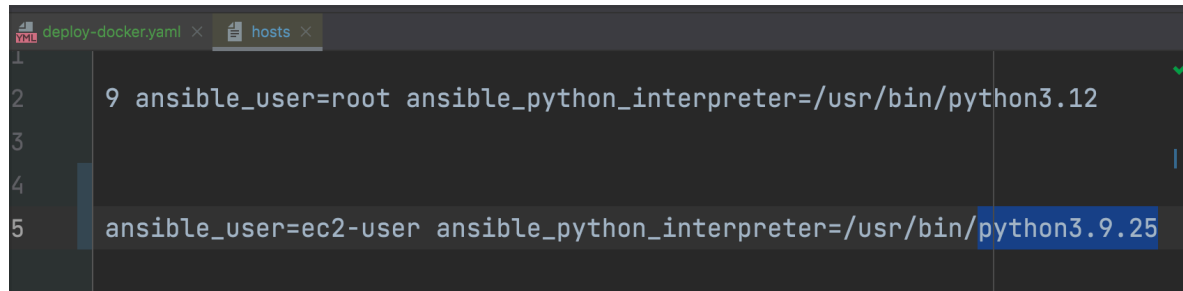
The following modules failed to execute: ansible.legacy.setup.

+---[ Sub-Event 1 of 1 ]---
|
| The module interpreter '/usr/bin/python3.12' was not found. Consider overriding the config value ansible_python_interpreter. See stdout/stderr for the returned output.
|
+---[ End Sub-Event ]---
```

I got an error related to python's version, so I checked the Ec2 python 3 version and is not 3.12, it is 3.9.25


```
[ec2-user@ip-10-0-10-56 ~]$ python3 --version
Python 3.9.25
[ec2-user@ip-10-0-10-56 ~]$
```

So I update the value in 'hosts' value

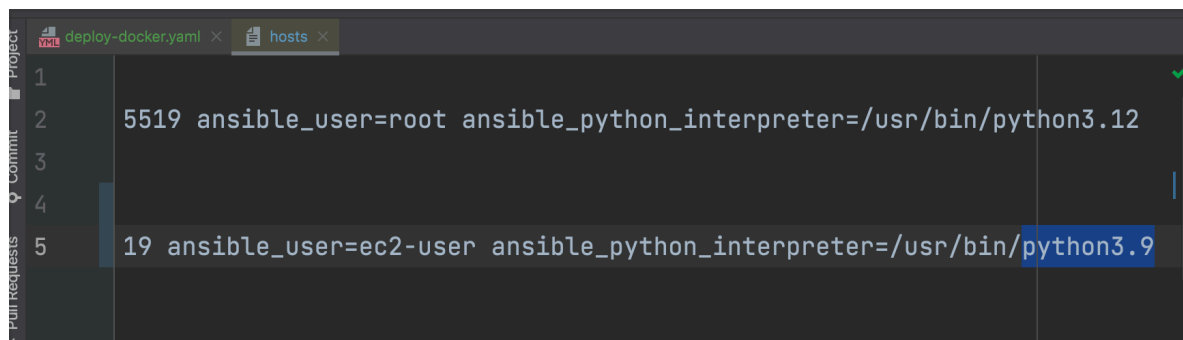


```
1 9 ansible_user=root ansible_python_interpreter=/usr/bin/python3.12
2
3
4
5 ansible_user=ec2-user ansible_python_interpreter=/usr/bin/python3.9.25
```

Same error. The problem is that I need the binary filename not the full version string (major.minor.patch).

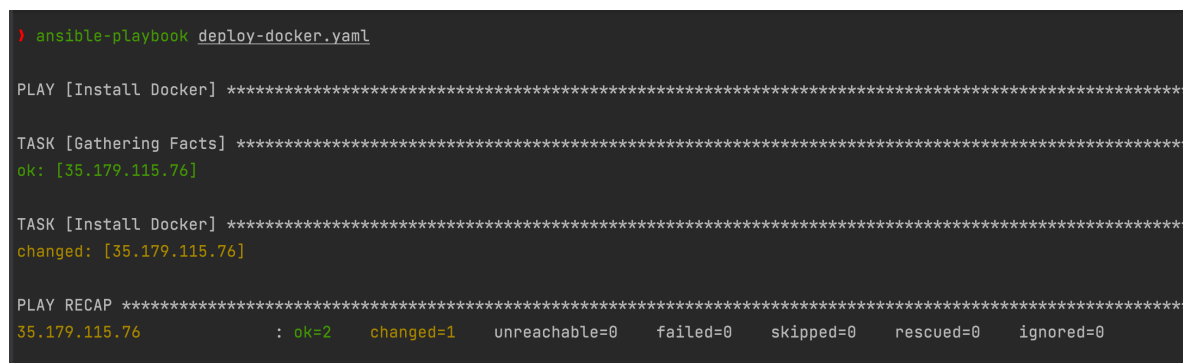
Binaries are only ever named after major.minor — python3.9, not python3.9.25. The patch version (25) isn't part of the executable's name on disk.

So I edit the 'hosts' file again to use 3.9 only



```
1 5519 ansible_user=root ansible_python_interpreter=/usr/bin/python3.12
2
3
4
5 19 ansible_user=ec2-user ansible_python_interpreter=/usr/bin/python3.9
```

And it works



```
> ansible-playbook deploy-docker.yaml

PLAY [Install Docker] *****

TASK [Gathering Facts] *****
ok: [35.179.115.76]

TASK [Install Docker] *****
changed: [35.179.115.76]

PLAY RECAP *****
35.179.115.76 : ok=2 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

Write 2nd play

Install docker-compose

So we want to start containers from the docker-compose file on the EC2 server and for that we need both docker and docker -compose

In EC2 we can't install docker compose using yum because docker compose is not a yum package **!!**

If we try to do it manually it fails:

```
[ec2-user@ip-10-0-10-186 ~]$ sudo yum install docker-compose
Last metadata expiration check: 1:00:15 ago on Thu Feb 15 09:21:52 2024.
No match for argument: docker-compose
Error: Unable to find a match: docker-compose
[ec2-user@ip-10-0-10-186 ~]$
```

In a previous demo we installed docker-compose directly into a EC2 server using a CI/CD pipeline in Module 12 - chapter 23

With similar steps to what we did in Module 9 - chapter 9

https://github.com/AstridCaballero/Module_9_Java-maven-app/blob/Module_9/chapter_9_Jenkins-Jobs_Docker_Compose/server-cmds.sh

The screenshot shows a Jenkins pipeline console output. On the left, a sidebar lists various modules and chapters. The main area displays the console output for 'Module 9 - chapter 9'. It shows a terminal session where the user attempts to install docker-compose using yum, which fails. The user then uses a script to download and install docker-compose manually. The script output shows the download progress and the installation of the docker-compose command. The user then runs 'docker-compose -f docker-compose.yaml up'.

```
Steps:
1. Install docker-compose on EC2 instance
2. Create a docker-compose.yaml file in the application
3. Adjust Jenkinsfile to execute docker-compose command on EC2 Instance

Install docker-compose on EC2 instance

Our EC2 instance has docker but it does not have docker-compose command installed yet:

[ec2-user@ip-172-31-45-189 ~]$ docker --version
Docker version 25.0.14, build 00d0007
[ec2-user@ip-172-31-45-189 ~]$ docker-compose
-bash: docker-compose: command not found
[ec2-user@ip-172-31-45-189 ~]$
```

So I use the code shared by Nana:

```
[ec2-user@ip-172-31-45-189 ~]$ docker --version
Docker version 25.0.14, build 00d0007
[ec2-user@ip-172-31-45-189 ~]$ docker-compose
-bash: docker-compose: command not found
[ec2-user@ip-172-31-45-189 ~]$ sudo curl -L https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname -s)-$(uname -m) -o /usr/local/bin/docker-compose
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
100 31621k 100 31621k  0  0  54626k  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
[ec2-user@ip-172-31-45-189 ~]$
```

And now the command binaries are stored locally and the executable is located here

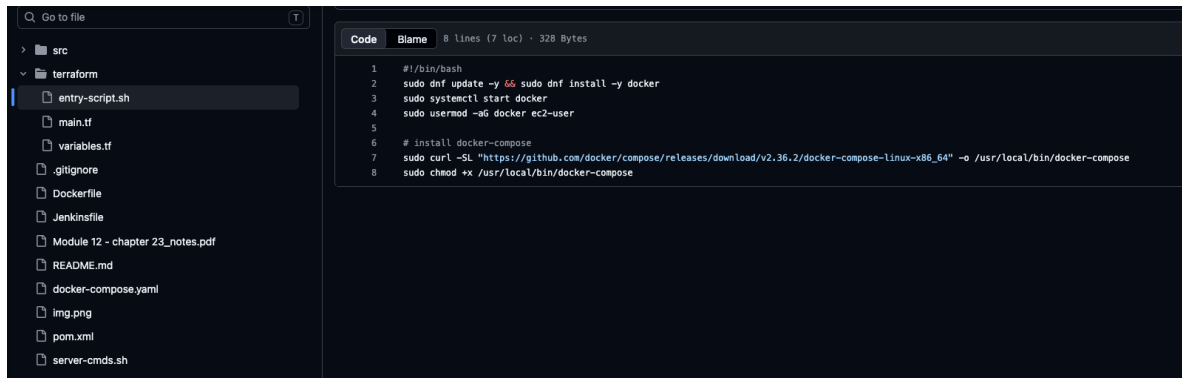
```
-> /usr/local/bin/docker-compose
```

But the executable is not executable just yet, so from the documentation we need to run

```
-> sudo chmod +x /usr/local/bin/docker-compose
```

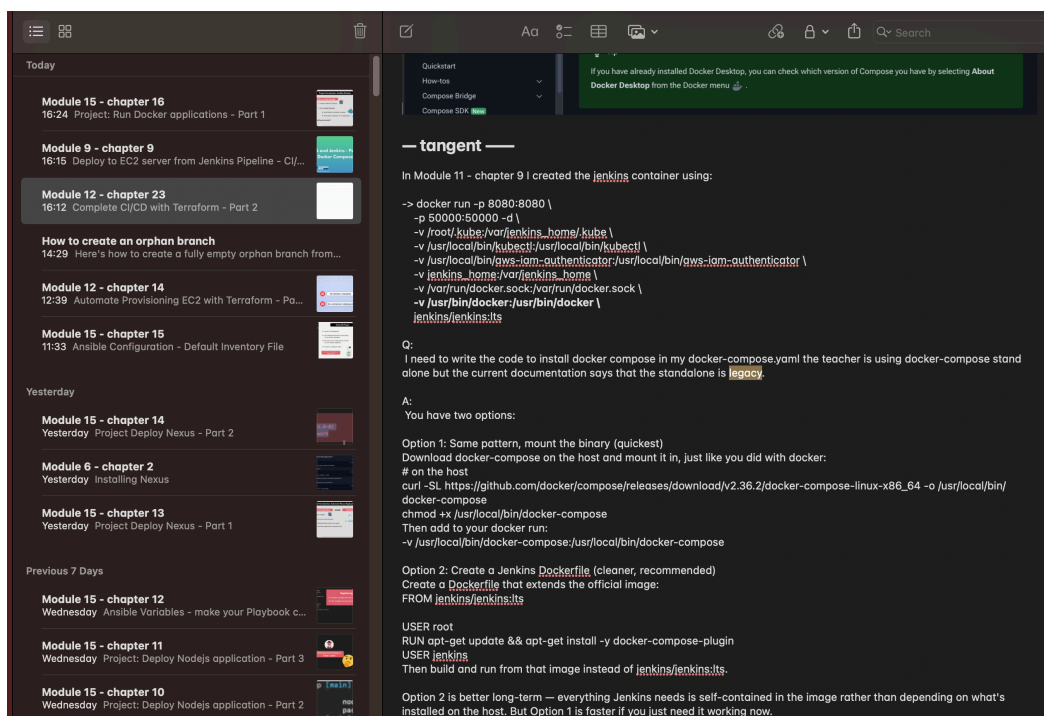
Then in Module 12 - chapter 23 we used an entry-script.sh that was executed in a fresh new EC2 instance that we provisioned with terraform.

https://github.com/AstridCaballero/Module_12_Terraform/blob/Module_12/Terraform_chapter-23/terraform/entry-script.sh



```
1 #!/bin/bash
2 sudo dnf update -y && sudo dnf install -y docker
3 sudo systemctl start docker
4 sudo usermod -aG docker ec2-user
5
6 # Install docker-compose
7 sudo curl -SL "https://github.com/docker/compose/releases/download/v2.36.2/docker-compose-linux-x86_64" -o /usr/local/bin/docker-compose
8 sudo chmod +x /usr/local/bin/docker-compose
```

And Nana installed docker-compose as a stand alone installation but in Module 12 - chapter 23 I couldn't because at the time I did the demo it was already legacy as seen in the notes:



— tangent —

In Module 11 - chapter 9 I created the [jenkins](#) container using:

```
-> docker run -p 8080:8080 \
  -p 50000:50000 -d \
  -v /root/.kube:/var/jenkins_home/.kube \
  -v /usr/local/bin/kubectl:/usr/local/bin/kubectl \
  -v /usr/local/bin/aws-iam-authenticator:/usr/local/bin/aws-iam-authenticator \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v /usr/bin/docker:/usr/bin/docker \
  jenkins/jenkins:its
```

Q:
I need to write the code to install docker compose in my docker-compose.yml the teacher is using docker-compose stand alone but the current documentation says that the standalone is **legacy**.

A:
You have two options:

Option 1: Same pattern, mount the binary (quickest)
Download docker-compose on the host and mount it in, just like you did with docker:
on the host
curl -SL https://github.com/docker/compose/releases/download/v2.36.2/docker-compose-linux-x86_64 -o /usr/local/bin/docker-compose
chmod +x /usr/local/bin/docker-compose
Then add to your docker run:
-v /usr/local/bin/docker-compose:/usr/local/bin/docker-compose

Option 2: Create a Jenkins **Dockerfile** (cleaner, recommended)
Create a **Dockerfile** that extends the official image:
FROM [jenkins/jenkins:its](#)

USER root
RUN apt-get update && apt-get install -y docker-compose-plugin
USER jenkins
Then build and run from that image instead of [jenkins/jenkins:its](#).

Option 2 is better long-term — everything Jenkins needs is self-contained in the image rather than depending on what's installed on the host. But Option 1 is faster if you just need it working now.

Now for this demo, Nana won't use the stand alone installation but the cli plugin installation for docker-compose which works more intuitively with Ansible's docker modules.

So these are the commands with the addition of line 6 and the editions of some paths from Nana for this demo that we need to translate to ansible:

```
hosts M ! deploy-docker.yaml U $ #!/bin/bash Untitled-1 ...
1 #!/bin/bash
2 sudo yum update -y && sudo yum install -y docker
3 sudo systemctl start docker
4 sudo usermod -aG docker ec2-user
5
6 mkdir -p ~/.docker/cli-plugins
7 curl -SL "https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname -s)-$(uname -m)"
8 chmod +x ~/.docker/cli-plugins/docker-compose
```

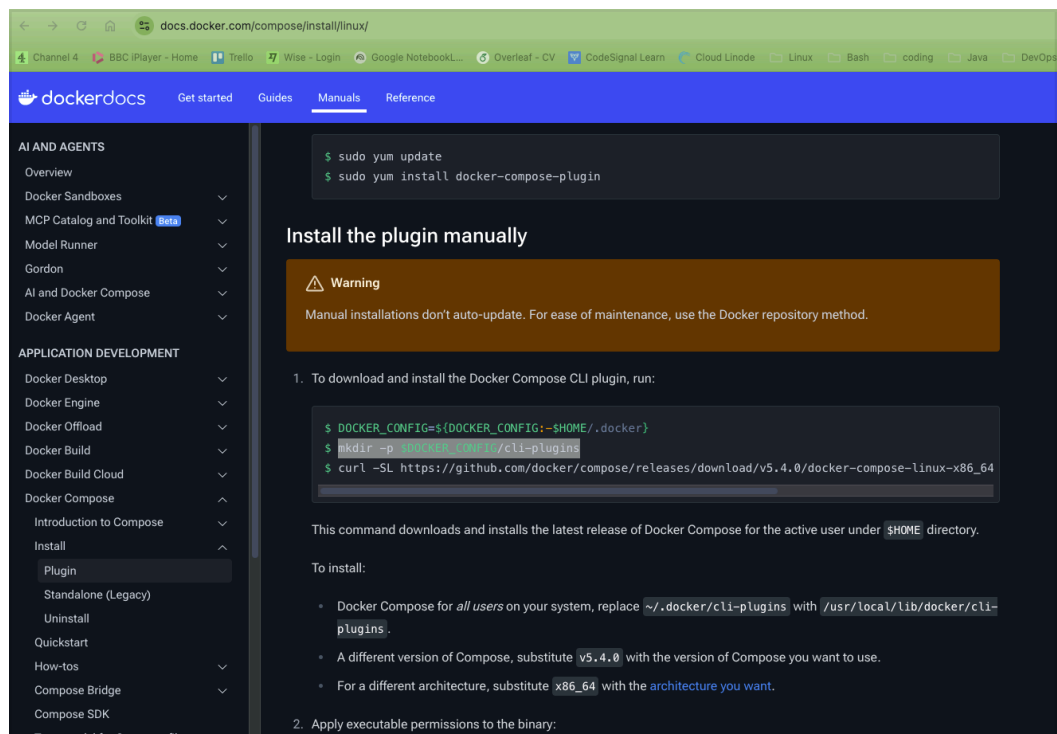
Line 7 🙌 is long, here is the rest:

```
hosts M ! deploy-docker.yaml U $ #!/bin/bash Untitled-1 ...
1
2 cker
3
4
5
6
7 releases/latest/download/docker-compose-$(uname -s)-$(uname -m) -o ~/.docker/cli-plugins/docker-compose
8 se
```

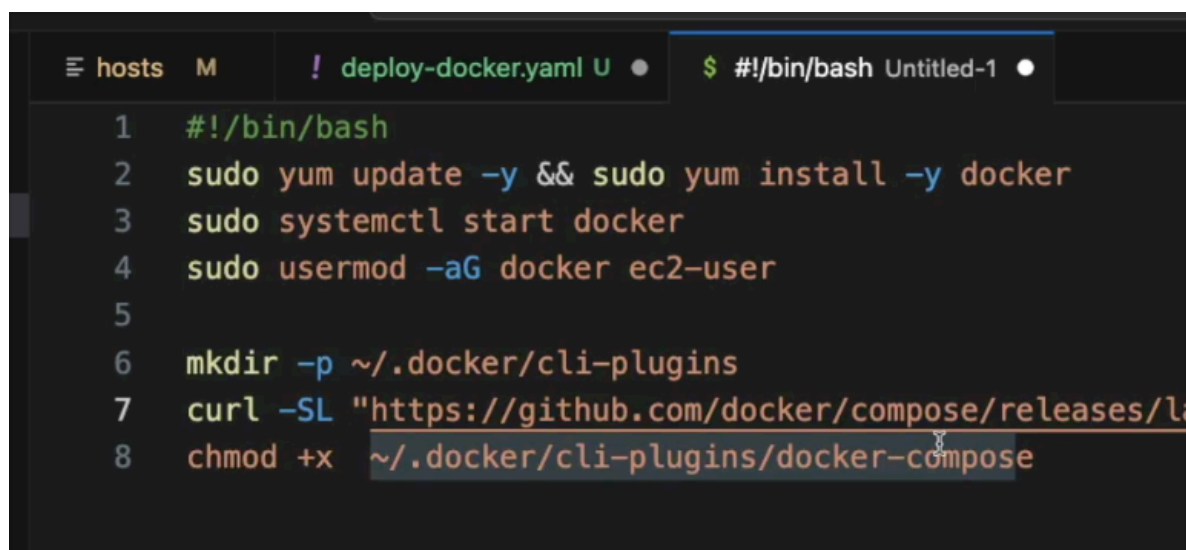
And Nana is following the plugin installation manually, and for that we need to create a directory Called 'cli-plugins' first which is in line 6 of her shell file above.

<https://docs.docker.com/compose/install/linux/>

We need this folder to download docker-compose in that location as per line 7 in the shell file



And finally we need to make the docker-compose file executable as per line 8 in the shell file.



Now let's map the shell commands into an ansible play:

```

12  - name: Install Docker-compose
13      hosts: docker_server
14      tasks:
15          - name: Create docker-compose directory
16              file:

```

And we will use the 'file' module to create the new directory

From the documentation we know that the new folder needs to be created inside the /.docker folder

```

$ DOCKER_CONFIG=${DOCKER_CONFIG:-$HOME/.docker}
$ mkdir -p $DOCKER_CONFIG/cli-plugins

```

And the folder name will be 'cli-plugins'

```

12  - name: Install Docker-compose
13      hosts: docker_server
14      tasks:
15          - name: Create docker-compose directory
16              file:
17                  path: ~/.docker/cli-plugins
18                  state: directory

```

Now let's download from internet and install docker-compose so we will use 'get_url' module

Nana uses this url:

```

19  - name: Install docker-compose
20      get_url:
21          url: https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname -s)-$(uname -m)
22

```

So I compared with the documentation and it looks a bit different:

```
R_CONFIG=${DOCKER_CONFIG:-$HOME/.docker}
-p $DOCKER_CONFIG/cli-plugins
-SL https://github.com/docker/compose/releases/download/v5.4.0/docker-compose-linux-x86_64 -
```

And AI says:
here's what's actually going on:

Screenshot from Nana's code: grabs the latest release and tries to build the filename dynamically:

```
.../releases/latest/download/docker-compose-$(uname -s)-$(uname -m) -o  
~/.docker/cli-plugins/docker-compose
```

Screenshot from docs: pins an exact version and hardcodes the OS/arch:
.../compose/releases/download/v5.4.0/docker-compose-linux-x86_64 -o ...

The dynamic version is the "official-looking" pattern people copy around, but there's a real gotcha: Docker Compose v2's release assets use lowercase OS names (docker-compose-linux-x86_64), while `uname -s` on Linux returns Linux with a capital L. So `$(uname -s)` produces Linux, not linux, and the built URL won't match any actual file on GitHub — it'll 404.

Worse: if your curl command doesn't include `-f` (fail on HTTP error), curl will happily save the GitHub 404 error page as if it were the binary. It'll "succeed" silently, and then docker compose version will fail or error with something cryptic later — much harder to debug than an obvious curl failure.

Two ways to fix it if you want to keep the dynamic version:

```
curl -SL https://github.com/docker/compose/releases/latest/download/docker-  
compose-$(uname -s | tr '[:upper:]' '[:lower:]')-$(uname -m) -o ~/.docker/cli-  
plugins/docker-compose
```

Or just be explicit like Screenshot from docs does — hardcode linux-x86_64 (fine since you know your EC2 target is x86_64 Linux) and pin a version for reproducibility. That's honestly the safer choice for a provisioning script: predictable, reproducible, no silent case-mismatch failures.

So I use the hardcoded url provided by the documentation

```

12 - name: Install Docker-compose
13   hosts: docker_server
14   tasks:
15     - name: Create docker-compose directory
16       file:
17         path: ~/.docker/cli-plugins
18         state: directory
19     - name: Install docker-compose
20       get_url:
21         url: https://github.com/docker/compose/releases/download/v5.4.0/docker-compose-linux-x86_64
22         dest: ~/.docker/cli-plugins/docker-compose
23         mode: +x

```

And the mode we set it to executable.

uname is a command:

uname

► A command-line utility that prints basic information about the operating system name and system hardware

We execute them in the terminal of the server and we can get:

```

[ec2-user@ip-10-0-10-186 ~]$ uname -s
Linux
[ec2-user@ip-10-0-10-186 ~]$ uname -m
x86_64
[ec2-user@ip-10-0-10-186 ~]$

```

We know `uname -m` is `x86_64` which is the architecture of the server as the AI mentioned before, but if we don't then we can actually create a task before installing docker-compose task and store it in a variable and the use that variable


```

12 - name: Install Docker-compose
13   hosts: docker_server
14   tasks:
15     - name: Create docker-compose directory
16       file:
17         path: ~/.docker/cli-plugins
18         state: directory
19     - name: get architecture of remote machine
20       shell: uname -m
21       register: remote_arch
22     - name: Install docker-compose
23       get_url:
24         url: https://github.com/docker/compose/releases/download/v5.4.0/docker-compose-linux-{{remote_arch.stdout}}
25         dest: ~/.docker/cli-plugins/docker-compose
26         mode: +x

```

Pattern	Behavior
/releases/download/v5.4.0/docker-compose-linux-x86_64	Pinned – always fetches that exact tagged release
/releases/latest/download/docker-compose-linux-x86_64	Dynamic – redirects to whatever the current latest release is

So if you want "always latest," drop the version segment entirely:

```
https://github.com/docker/compose/releases/latest/download/docker-compose-linux-{{ remote_arch.stdout }}
```

If you want reproducible/pinned installs (recommended for provisioning scripts — so a fresh EC2 build next month doesn't silently pick up a newer, possibly breaking, Compose version), keep v5.4.0 hardcoded as you have it.

So I decided to create a variable to hold the version of docker-compose

```
12  name: Install Docker-compose
13  hosts: docker_server
14  vars_files:
15    project-vars
16  tasks:
17    - name: Create docker-compose directory
18      file:
19        path: ~/.docker/cli-plugins
20        state: directory
21    - name: get architecture of remote machine
22      shell: uname -m
23      register: remote_arch
24    - name: Install docker-compose
25      get_url:
26        url: https://github.com/docker/compose/releases/download/v{{docker_compose_version}}/docker-compo

Document 1/1  Item 2/2  tasks:  Item 3/3  get_url:  url: https://github.com/d...

project-vars
1  version: 1.0.0
2  location: /Users/astrid/PythonProject/Ansible/nodejs-app
3  linux_name: astrid
4  user_home_dir: /home/{{linux_name}}
5
6  nexus_version: 3.84.1-01
7  docker_compose_version: 5.4.0
```

See I also had to add lines 14 and 15 🙌

Run the playbook

-> ansible-playbook deploy-docker.yaml

```
TASK [Gathering Facts] *****
ok: [35.179.115.76]

TASK [Create docker-compose directory] *****
changed: [35.179.115.76]

TASK [get architecture of remote machine] *****
changed: [35.179.115.76]

TASK [Install docker-compose] *****
changed: [35.179.115.76]

PLAY RECAP *****
35.179.115.76      : ok=6    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

And if I check the server

```
[ec2-user@ip-10-0-10-56 ~]$ docker compose
Usage:  docker compose [OPTIONS] COMMAND

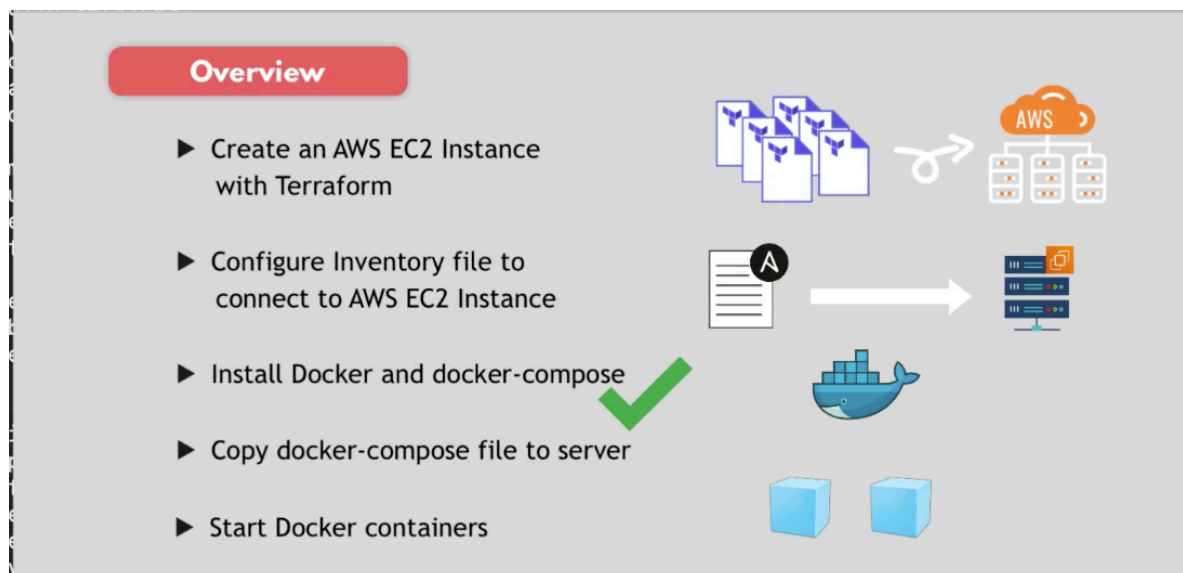
Define and run multi-container applications with Docker

Options:
  --all-resources          Include all resources, even those not
  --ansi string            Control when to print ANSI control cha
  --compatibility          Run compose in backward compatibility
  --dry-run               Execute command in dry run mode
  --env-file stringArray  Specify an alternate environment file
-f, --file stringArray    Compose configuration files
  --parallel int          Control max parallelism, -1 for unlimi
  --profile stringArray   Specify a profile to enable
  --progress string       Set type of progress output (auto, tty
  --project-directory string Specify an alternate working directory
                           (default: the path of the, first speci
  -p, --project-name string Project name

Management Commands:
  bridge                  Convert compose files into another model

Commands:
  attach                  Attach local standard input, output, and error
  build                   Build or rebuild services
  commit                  Create a new image from a service container's c
```

Now we have the command available



From this commands:

```
Code Blame 8 lines (7 loc) · 328 Bytes
1  #!/bin/bash
2  sudo dnf update -y && sudo dnf install -y docker
3  sudo systemctl start docker
4  sudo usermod -aG docker ec2-user
5
6  # install docker-compose
7  sudo curl -SL "https://github.com/docker/compose/releases/download/v2.36.2/docker-compose-linux-x86_64" -o /usr/local/bin/docker-compose
8  sudo chmod +x /usr/local/bin/docker-compose
```

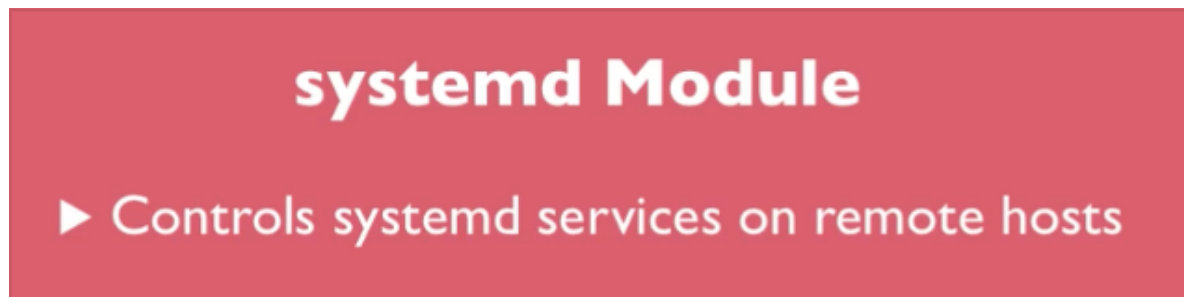
We have line 2, 7, 8. But docker is not running yet, so we need to map line 3.

```
[ec2-user@ip-10-0-10-56 ~]$ docker pull redis
Using default tag: latest
Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the docker daemon running?
[ec2-user@ip-10-0-10-56 ~]$
```

Write 3rd play

Start docker daemon

And we will need root user to run docker and the module 'systemd' module to map 'systemctl'



And the play looks like this

```

30  - name: Start docker
31      hosts: docker_server
32      become: yes
33      tasks:
34          - name: Start docker daemon
35              systemd:
36                  name: docker
37                  state: started

```

Execute the playbook

-> ansible-playbook deploy-docker.yaml

```

TASK [Gathering Facts] *****
ok: [35.179.115.76]

TASK [Start docker daemon] *****
changed: [35.179.115.76]

PLAY RECAP *****
35.179.115.76      : ok=8    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

```

And I check in the Ec2 instance

```

[ec2-user@ip-10-0-10-56 ~]$ docker pull redis
Using default tag: latest
Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the docker daemon running?
[ec2-user@ip-10-0-10-56 ~]$ docker pull redis
Using default tag: latest
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Post "http://%2Fvar%2Frun%2Fdocker.sock/v1.44/images/create?fromImage=redis&tag=latest": dial unix /var/run/docker.sock: connect: permission denied
[ec2-user@ip-10-0-10-56 ~]$

```

We can see docker is running but we have an error, a different error, we are having now permission issues. We should use sudo

-> sudo docker pull redis

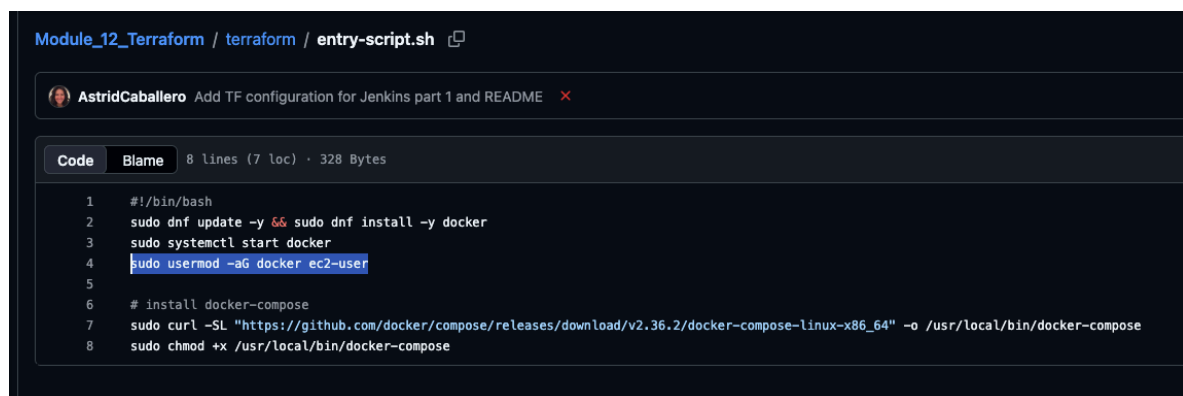
```
[ec2-user@ip-10-0-10-186 ~]$ sudo docker pull redis
Using default tag: latest
latest: Pulling from library/redis
e1caac4eb9d2: Extracting [====>] 2.064MB/29.12MB
7469c6c5b625: Download complete
a3d1b68c4a62: Download complete
152cbe749752: Download complete
7218480dfba1: Downloading [====>] 2.149MB/20.83MB
e61c48a0d344: Waiting
4f4fb700ef54: Waiting
82adb0efabd8: Waiting
```

It will work using 'sudo' but not for the ec2-user because the user executing a docker command needs to be part of 'docker' group and ec2-user is not part of that group yet.

Write 4th play

Add ec2-user to docker group

So we need to map line 4 to ansible



```
Module_12_Terraform / terraform / entry-script.sh
AstridCaballero Add TF configuration for Jenkins part 1 and README
Code Blame 8 lines (7 loc) · 328 Bytes
1 #!/bin/bash
2 sudo dnf update -y && sudo dnf install -y docker
3 sudo systemctl start docker
4 sudo usermod -aG docker ec2-user
5
6 # install docker-compose
7 sudo curl -SL "https://github.com/docker/compose/releases/download/v2.36.2/docker-compose-linux-x86_64" -o /usr/local/bin/docker-compose
8 sudo chmod +x /usr/local/bin/docker-compose
```

To add ec2-user to the group 'docker'
-aG -> add to group

Once we add ec2-user to the group 'docker' we can execute docker commands without 'sudo' keyword

The new play needs to use 'root' user to add another user to a group

```

39  - name: Add ec2-user to docker group
40      hosts: docker_server
41      become: yes
42      tasks:
43      - name: Add ec2-user to docker group
44        user:

```

And we will use the 'user' module. With this module we can create a new user and also add a user to a group or groups.

Currently ec2-user belong to 4 groups:

```

[ec2-user@ip-10-0-10-56 ~]$ groups
ec2-user adm wheel systemd-journal
[ec2-user@ip-10-0-10-56 ~]$ █

```

So we want to append 'docker' group to the list of groups that the user belongs to:

```

39  - name: Add ec2-user to docker group
40      hosts: docker_server
41      become: yes
42      tasks:
43      - name: Add ec2-user to docker group
44        user:
45          name: ec2-user
46          groups: docker
47          append: yes|

```

Manually we test the command
-> sudo usermod -aG docker ec2-user


```

[ec2-user@ip-10-0-10-186 ~]$ sudo gpasswd -d ec2-user docker
Removing user ec2-user from group docker
[ec2-user@ip-10-0-10-186 ~]$ groups
ec2-user adm wheel systemd-journal docker
[ec2-user@ip-10-0-10-186 ~]$ exit
logout
Connection to 3.75.173.238 closed.
nana@macbook /Users/nana
% ssh ec2-user@3.75.173.238

      #_
    _\  #####_      Amazon Linux 2023
   ~\  \#####\
   ~\  \####|
   ~\  \#/  ___  https://aws.amazon.com/linux/amazon-linux-2023
   ~\  V~'  '->
   ~~~
   ~~.~.~
   ~\  /  /
   ~\m/  '

Last login: Thu Feb 15 10:40:13 2024 from 85.246.35.188
[ec2-user@ip-10-0-10-186 ~]$ groups
ec2-user adm wheel systemd-journal
[ec2-user@ip-10-0-10-186 ~]$

```

And creates a new play to test and show that we will have the problem

```

37   - name: Add ec2-user to docker group
38     hosts: docker_server
39     become: yes
40     tasks:
41       - name: Add ec2-user to docker group
42         user:
43           name: ec2-user
44           groups: docker
45           append: yes
46
47   - name: Test docker pull
48     hosts: docker_server
49     tasks:
50       - name: Pull redis
51         command: docker pull redis

```

And she gets the permission error:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
fatal: [3.75.173.238]: FAILED! => ("changed": true, "cmd": ["docker", "pull", "redis"], "delta": "0:00:00.041888", "end": "2024-02-15 10:42:47.111192", "msg": "non-zero return code", "rc": 1, "start": "2024-02-15 10:42:47.069304", "stderr": "permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Post \"http://%2Fvar%2Frun%2Fdocker.sock/v1.24/images/create?fromImage=redis&tag=latest\": dial unix /var/run/docker.sock: connect: permission denied", "stdout_lines": ["permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Post \"http://%2Fvar%2Frun%2Fdocker.sock/v1.24/images/create?fromImage=redis&tag=latest\": dial unix /var/run/docker.sock: connect: permission denied"], "stdout": "Using default tag: latest", "stdout_lines": ["Using default tag: latest"]}

PLAY RECAP *****
3.75.173.238 : ok=11 changed=2 unreachable=0 failed=1 skipped=0 rescued=0 ignored=0

nana@macbook /Users/nana/ansible [master]
```

Ansible has a module called 'meta' to exit the session and reconnect to a new one:

```
39  - name: Add ec2-user to docker group
40      hosts: docker_server
41      become: yes
42      tasks:
43          - name: Add ec2-user to docker group
44              user:
45                  name: ec2-user
46                  groups: docker
47                  append: yes
48          - name: Reconnect to server session
49      meta: reset_connection
50
51  - name: test docker pull
52      hosts: docker_server
53      tasks:
54          - name: Pull redis
55              command: docker pull redis
```

Now execute the playbook:

-> ansible-playbook deploy-docker.yaml

```

TASK [Gathering Facts] *****
ok: [35.179.115.76]

TASK [Add ec2-user to docker group] *****
changed: [35.179.115.76]

TASK [Reconnect to server session] *****

PLAY [test docker pull] *****

TASK [Gathering Facts] *****
ok: [35.179.115.76]

TASK [Pull redis] *****
changed: [35.179.115.76]

PLAY RECAP *****
35.179.115.76      : ok=12  changed=3  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

```

No errors this time 

And check the server

-> docker images | grep redis

```

bash: redis: command not found
[ec2-user@ip-10-0-10-56 ~]$ docker images | grep redis
redis          latest          0458cdd27215    2 days ago     146MB
[ec2-user@ip-10-0-10-56 ~]$ █

```

Refactor the playbook

In Ansible the standard is to name the tasks with a sentence that reflects the outcome of the task.

So instead of 'Start docker daemon'

```
YML deploy-docker.yaml x
31     hosts: docker_server
32     become: yes
33     tasks:
34     - name: Start docker daemon
35       systemd:
36         name: docker
37         state: started
38
```

Is better 'Ensure Docker is running' as it will match the state: started

```
30 - name: Start docker
31     hosts: docker_server
32     become: yes
33     tasks:
34     - name: Ensure Docker is running
35       systemd:
36         name: docker
37         state: started
```

Same for 'Install Docker'

```
1  ---
2  - name: Install Docker
3    hosts: docker_server
4    become: yes
5    tasks:
6  - name: Install Docker
7    yum:
8      name: docker
9      update_cache: yes
10     state: present
11
```

Is better 'Make sure Docker is installed'

```
1  ---
2  - name: Install Docker
3    hosts: docker_server
4    become: yes
5    tasks:
6  - name: Make sure Docker is installed
7    yum:
8      name: docker
9      update_cache: yes
10     state: present
```

We can also can start the docker daemon after we installed docker and it also make sense to have them in the same play as both need root user, so the two below

```
1  -
2  - name: Install Docker
3    hosts: docker_server
4    become: yes
5    tasks:
6      - name: Make sure Docker is installed
7        yum:
8          name: docker
9          update_cache: yes
10         state: present
11
12  + - <4 keys>
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30  - name: Start docker
31    hosts: docker_server
32    become: yes
33    tasks:
34      - name: Ensure Docker is running
35        systemd:
36          name: docker
37          state: started
38
```

Become one play

```
1  ---
2  - name: Install Docker
3    hosts: docker_server
4    become: yes
5    tasks:
6  - name: Make sure Docker is installed
7    yum:
8      name: docker
9      update_cache: yes
10     state: present
11  - name: Ensure Docker is running
12    systemd:
13      name: docker
14      state: started
15
16  - <4 keys>
33  #
34  #- name: Start docker
35  #   hosts: docker_server
36  #   become: yes
37  #   tasks:
38
```